

# 07 · Moderne CPU-Architekturen

Mikrocomputertechnik 3 · Von Pipelining-Limits über Out-of-Order zur Branch Prediction

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 67

## Willkommen zu Block 7!

- Willkommen zur siebten Vorlesung in Mikrocomputertechnik 3
- Bisher: RAM, Caches, Interrupts, Busse, DMA — das SoC ist ausgestattet
- Peripherie und Transfers können autonom laufen; der Engpass wandert zurück zum Kern
- Heute: wie High-End-CPU's intern mehr Rechenleistung aus demselben Takt holen

MCT1: 5-Stufen-CPU



MCT3: SoC-Peripherie



Heute: High-End-Kern

## Notizen

Caches und DMA haben externe Engpässe entschärft. Zurück zum Prozessorkern: Warum reicht die MCT1-Pipeline für Desktop-/Smartphone-CPU's nicht?

Folie

2 / 67

## Anknüpfung: DMA-Labor

Kurz-Recap aus Block 6 (Vertiefung im Übungsblatt):

- DMA kopiert Arrays autonom; Polling blockiert die CPU stark
- Mit DMA bleibt die Hauptschleife (z. B. LED) oft reaktionsfähig
- Bus-Mastering und Cycle Stealing: DMA teilt sich den Bus mit der CPU
- Folge: weniger ALU-Zeit für stupides Kopieren — mehr Zeit für echte Berechnung

```
/* Konzept - Adressen/Bits fiktiv bzw. Datenblatt */  
DMA_CH1->SRC_ADDR = (uint32_t)text_array;  
DMA_CH1->DST_ADDR = (uint32_t)&UART_TX;  
DMA_CH1->CONTROL |= DMA_ENABLE; /* CPU kehrt sofort zurück */
```

## Notizen

DMA befreit von Kopierarbeit. Als Nächstes: Wie schnell rechnet die CPU selbst auf Befehlsebene?

## Folie

3 / 67

### Zurück in die CPU

- Fokus: Ausführungsgeschwindigkeit im Kern
- Weg von Systembus, MMIO und Peripherie-IRQs
- Rein: Pipeline, ALU, Assembler auf Befehlsebene
- Metriken und Hardware-Parallelität innerhalb eines Kerns

## Notizen

Alles außerhalb des L1 blenden wir didaktisch aus — Takte, Gatter, Instruktionsstrom.

## Folie

4 / 67

### Refresher MCT1: 5-Stufen-Pipeline

Klassische RISC-Pipeline (Harris/Harris):

| Stufe | Name              | Aufgabe                    |
|-------|-------------------|----------------------------|
| IF    | Instruction Fetch | Befehl holen               |
| ID    | Decode            | Dekodieren, Register lesen |
| EX    | Execute           | ALU                        |
| MEM   | Memory            | Datenspeicher              |
| WB    | Write Back        | Ergebnis ins Registerfile  |

Idealfall nach Füllen: ein abgeschlossener Befehl pro Takt.

## Notizen

Stand MCT1: Fließband mit fünf Stufen — Grundlage für alles Weitere heute.

### Metriken: IPC und CPI

- IPC (Instructions Per Cycle): abgeschlossene Befehle pro Takt (höher besser)
- CPI (Cycles Per Instruction): Takte pro Befehl im Mittel (niedriger besser)
- Kehrwerte:  $CPI = 1/IPC$
- Laufzeit:

$$T = N_{\text{Instr}} \times CPI \times T_{\text{Takt}}$$

### Notizen

Drei Hebel: weniger Befehle, kürzerer Takt, bessere Architektur (kleineres CPI).

### Limit der skalaren Pipeline

- Ideal ohne Stalls/Hazards/Misses:  $IPC = 1$ ,  $CPI = 1$
- Eine skalare Ein-Issue-Pipeline: strukturell kein  $CPI < 1$
- Ein Pfad → Befehle im Gänsemarsch
- Realität oft schlechter ( $CPI > 1$ ) durch Bubbles und Hazards

### Notizen

Mauer der MCT1-Architektur: IPC höchstens 1 im Ideal — oft darunter.

### Die Taktraten-Mauer (Power Wall)

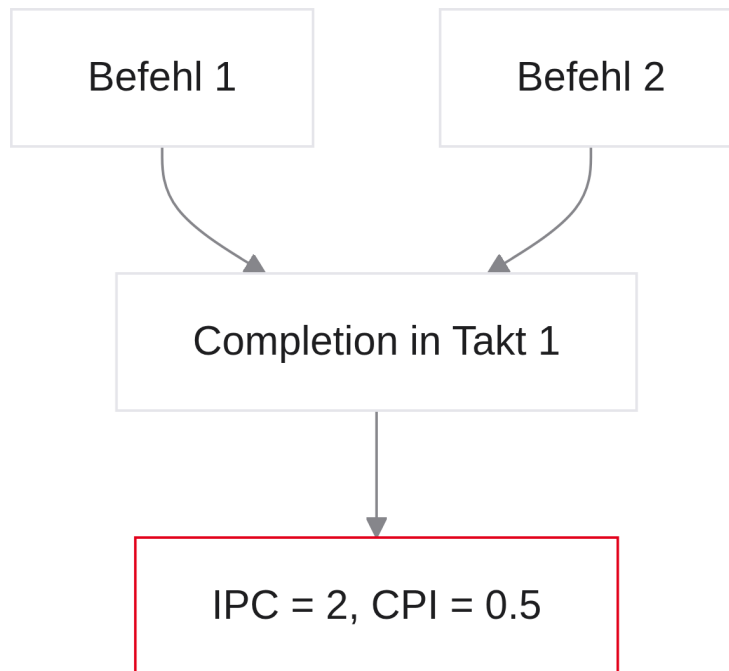
- Idee der 2000er: Frequenz massiv erhöhen (GHz-Wettrennen)
- Dynamische Verlustleistung und Leckströme begrenzen den sinnvollen Takt
- Typisch: Desktop-/Mobile-Kerne oft im Bereich weniger GHz — nicht „unendlich“
- CPI/IPC verbessern, statt nur höher zu takten

### Notizen

Pentium-4-Ära: Hoffnung auf extrem hohe GHz — Physik und Kühlung setzen Grenzen.

## Die Leitfrage des Tages

- Taktfrequenz: physikalisch begrenzt
- Skalare Pipeline: Ideal  $CPI = 1$  (mit Stalls oft  $> 1$ )
- Wie erreicht man  $CPI < 1$  (mehrere Completions pro Takt) bei gleichem Takt?
- Antwort: Parallelität *innerhalb* des Kerns (ILP ausnutzen)



## Notizen

Nicht schneller takten — breiter und smarter schedulen.

## Agenda

1. Grenzen des Pipelining
2. ILP und Superskalarität
3. Out-of-Order Execution
4. Branch Prediction
5. Spekulation und Security (Spectre/Meltdown)
6. Labor-Ausblick (Compiler / Godbolt)

## Notizen

Von skalarer Pipeline über OoO und Prediction bis Spekulation und Compiler.

## Lernziele Block 7

- Skalare vs. superskalare Pipeline erklären
- Datenabhängigkeiten RAW, WAW, WAR unterscheiden
- Register Renaming und Reorder Buffer (ROB) einordnen
- Warum Branches tiefe Pipelines treffen
- Rolle von Compiler-Scheduling (–03)

## Notizen

Ziel: Datenblätter moderner Kerne architektonisch lesen können — nicht nur Takt und Kerne zählen.

## Instruction-Level Parallelism (ILP)

- ILP: wie viele Befehle eines Programms *theoretisch* parallel laufen könnten
- Beispiel ohne Abhängigkeit:

```
a = b + c;  
x = y * z;
```

- Unabhängige Operationen → auf getrennten Execution Units im selben Takt denkbar

## Notizen

Viele Programme haben unabhängige Teilrechnungen — das ist der Hebel für  $IPC > 1$ .

## Datenabhängigkeiten: RAW

- True Dependency / Read-After-Write (RAW): Befehl 2 braucht Ergebnis von Befehl 1

```
add t0, t1, t2    # schreibt t0  
sub t3, t0, t4    # liest t0 – muss warten
```

- Keine Parallelität dieser beiden im selben EX-Schritt — ILP dieses Paares = 1 (rein sequentiell)

## Notizen

Bekannt aus MCT1: echte Datenabhängigkeit — Hardware kann Mathematik nicht „wegoptimieren“.

### Statisches vs. dynamisches Scheduling

- Statisch (Compiler): Befehle vorab umordnen, Abhängigkeiten auseinanderziehen
- Dynamisch (Hardware): zur Laufzeit unabhängige Befehle finden und an die Units ausgeben (Issue)
- Compiler kennt u. a. Cache-Misses zur Laufzeit nicht → dynamisches Scheduling oft mächtiger

#### Notizen

Beide Ansätze ergänzen sich; High-End-Kerne verlassen sich stark auf Hardware-Scheduling.

### Superskalare Architekturen

- Skalar: typisch ein Issue pro Takt
- Superskalar: Multiple Issue — mehrere Befehle starten (nahezu) im selben Takt die Execute-Phase
- Pipeline wird „breiter“: mehrspurige Autobahn statt einspurigem Fließband

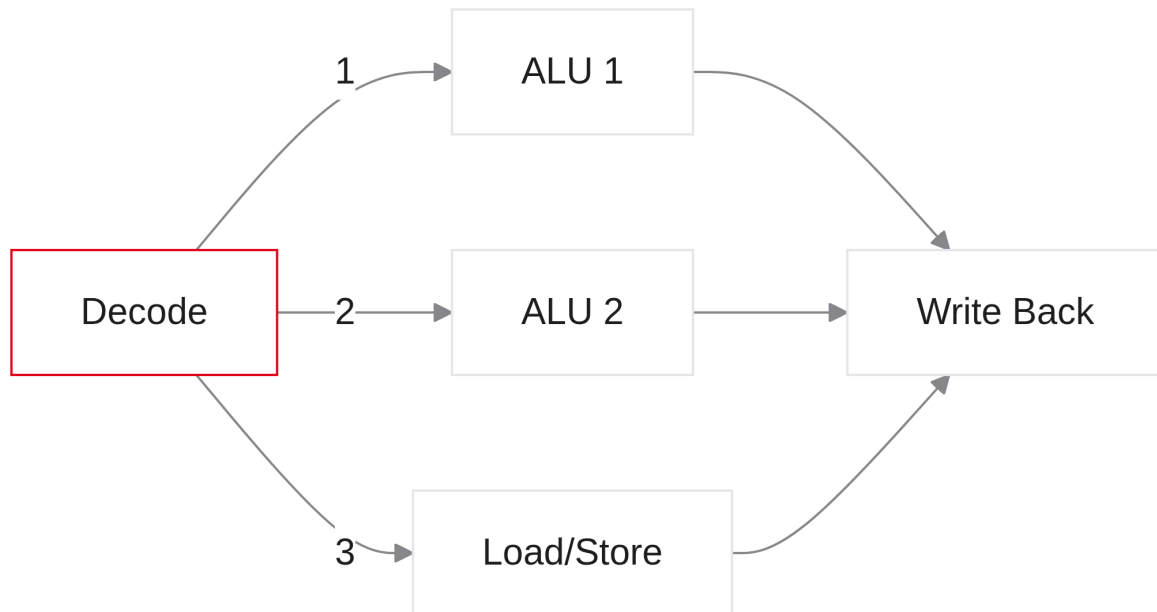
#### Notizen

Erster Schritt zu  $CPI < 1$ : Execution Units und Issue-Breite vervielfachen.

### Mehrere Execution Units

Beispiel Issue-Width  $\approx 3$ :

- ALU 1, ALU 2 (Integer/Logik)
- Load/Store-Unit



### Notizen

Drei unabhängige Befehle → drei Units → im Ideal drei Completions; CPI kann unter 1 sinken.

### Folie

16 / 67

### Multi-Issue Fetch und Decode

- Fetch holt mehrere Instruktionen pro Takt aus dem I-Cache (z. B. 16 Byte)
- Decode prüft Abhängigkeiten *zwischen* den Kandidaten und verteilt auf Units
- Aufwand der Abhängigkeitsprüfung wächst stark mit der Issue-Width

### Notizen

Breites Back-End nützt wenig ohne breites Front-End — Decode wird oft zum Flaschenhals.

### In-Order Superskalar

- Start-Reihenfolge = Programmreihenfolge (z. B. ältere In-Order-Kerne)
- Unabhängige Nachbarn können gemeinsam starten
- Stoppt ein früher Befehl (Hazard/Miss), stauen dahinter auch Unabhängige

#### Notizen

Breit, aber starr: ein Stau vorne blockiert freie Units hinten.

### Problem: Cache-Miss in-order

```
lw  t0, 0(s1)    # Miss → viele Wartezyklen
add t1, t0, t2    # RAW auf t0 → muss warten
sub t4, t5, t6    # unabhängig - trotzdem oft blockiert
```

- In-Order: unabhängiger sub wartet hinter dem Stau
- Units stehen ungenutzt — IPC bricht ein

#### Notizen

Cache-Misses sind Gift für strikte Programmreihenfolge trotz freier ALUs.

### Falsche Abhängigkeit: WAW

Write-After-Write:

```
add t0, t1, t2    # schreibt t0
sub t0, t4, t5    # schreibt ebenfalls t0
```

- Keine RAW-Abhängigkeit der Operanden
- Dennoch: Schreibreihenfolge muss erhalten bleiben — sonst falscher Endwert in t0

#### Notizen

Konflikt nur wegen wiederverwendeten Zielregisters — „falsche“ Abhängigkeit.



## Falsche Abhängigkeit: WAR

Write-After-Read:

```
add t1, t0, t2    # liest t0
sub t0, t4, t5    # überschreibt t0
```

- sub darf t0 nicht zu früh überschreiben, bevor add gelesen hat
- Verhindert unkritische Überholvorgänge ohne weitere Maßnahmen

### Notizen

Wieder Register-Recycling: mathematisch unabhängig, architektonisch verkoppelt.

## Engpass: 32 ISA-Register

- RISC-V: 32 Architektur-Register x0–x31
- Compiler muss Register stark wiederverwenden → viele WAW/WAR
- Künstliche Konflikte drosseln ausnutzbares ILP

### Notizen

ISA-Register sind die sichtbare Schnittstelle — zu wenige physische „Kisten“ für aggressives Scheduling.

## Fragen? — ILP und Superskalarität

- Wie hebt Superskalarität die IPC?
- Warum stoppt ein Cache-Miss eine In-Order-Pipeline oft global?
- RAW vs. WAW/WAR?

### Notizen

Als Nächstes: Out-of-Order — Staus umfahren, falsche Abhängigkeiten per Renaming auflösen.

## Out-of-Order Execution (OoO)

- Dynamisches Scheduling: blockierte Befehle „parken“
- Unabhängige spätere Befehle dürfen vorgezogen werden
- Ausführung asynchron, sobald Operanden bereit sind — nicht strikt nach Programmzeile

### Notizen

Modern: nicht mehr nur „nächster PC“ — freie Units mit bereiten Operanden füttern.

## Datenfluss-Maschine

- Instruktion „feuert“, wenn Operanden da sind — nicht nur wenn der PC sie erreicht
- Cache-Miss blockiert vor allem *abhängige* Nachfolger
- Unabhängiger Code kann am Stau vorbeilaufen

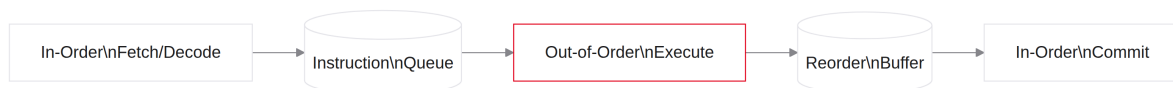
### Notizen

Paradigmenwechsel: vom starren Befehlszeiger zum datengetriebenen Issue.

## Drei Phasen der OoO-Pipeline

1. In-Order Front-End: Fetch / Decode
2. Out-of-Order Back-End: Execute
3. In-Order Retire (Commit): Ergebnisse programmgerecht sichtbar machen

Dazwischen: große Puffer (Queues, ROB).



### Notizen

Vorne geordnet lesen, Mitte ausnutzen, hinten Ordnung für Software/OS wiederherstellen.

## Front-End und Renaming

- Mehrere Befehle aus dem I-Cache holen und dekodieren
- Vor dem Chaos: Register Renaming gegen WAW/WAR
- Architektur-Register der ISA  $\neq$  physische Registerdatei

### Notizen

32 ISA-Namen bleiben die Illusion für den Compiler — Hardware mappt um.

## Register Renaming

- OoO-Kerne: großer Pool physischer Register (Größenordnung: oft  $> 100$ )
- Architektur-Register ( $x0-x31$ ) sind Namen / Mapping
- Jeder Befehl mit Zielregister bekommt typisch ein neues freies physisches Ziel

### Notizen

Genialer Trick: WAW/WAR verschwinden, weil Schreibziele physisch getrennt werden.

## Renaming am Beispiel (WAW)

```
add t0, t1, t2
sub t0, t4, t5
```

Hardware (schematisch):

- add  $\rightarrow$  schreibt P10 (Mapping  $t0 \rightarrow P10$ )
- sub  $\rightarrow$  schreibt P11 (neues Mapping  $t0 \rightarrow P11$ )

Kein gemeinsames physisches Ziel  $\rightarrow$  paralleles Issue möglich.

### Notizen

Namen werden recycelt, physische Kisten sind frisch — ILP gerettet.

### Reservation Stations

- Nach Renaming: Befehle in Reservation Stations / Issue Queue
- Puffer vor den Units (oft viele Dutzend bis  $> 100$  Einträge)
- Warten auf Operanden (physische Register / Tag-Match am Datenbus)

### Notizen

Wartehalle: Bordkarte = physische Register; Abflug = Operanden komplett.

### Out-of-Order Issue

- Ergebnisse erscheinen auf dem Common Data Bus / Wakeup-Netz
- Wartende Einträge greifen Werte ab; bereite Befehle melden sich
- Scheduler gibt bereite Befehle in freie Units — Reihenfolge  $\neq$  Programmzeile

### Notizen

Wer Daten hat, rechnet; wer auf Memory wartet, bleibt in der Queue.

### Problem: Interrupts im Chaos

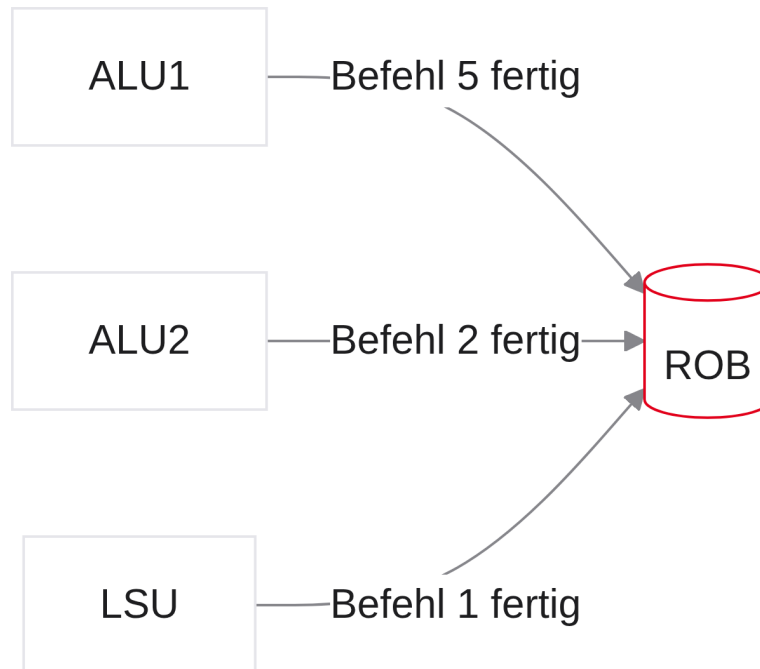
- Frühe Befehle noch nicht fertig, spätere schon executed
- Interrupt / Exception: architektonisch sichtbarer Zustand muss konsistent sein
- Ohne Schutz: RAM/Register könnten „Zukunft“ schon enthalten — inkonsistent für OS

### Notizen

Betriebssysteme erwarten präzise, geordnete Architektursicht — nicht Zwischenchaos.

### Reorder Buffer (ROB)

- ALU-/Load-Ergebnisse nicht sofort final sichtbar schreiben
- Zuerst in den ROB (Ringpuffer), sortiert nach ursprünglicher Programmordnung



#### Notizen

Netz mit doppeltem Boden: rechnen darf chaotisch sein — sichtbar werden geordnet.

### In-Order Commit (Retire)

- Commit nur am ältesten noch nicht retired Befehl
- Erst dann: finales Update der Architektur-Register / Stores
- Spätere fertige Einträge warten im ROB, bis die Vorgänger committed sind
- Nach außen: scheinbar sequentielle CPU

#### Notizen

Programmierer und OS sehen geordnete Semantik — trotz OoO im Back-End.

## Exceptions und ROB

- Commit stoppt am fehlerhaften / unterbrochenen Befehl
- Jüngere ROB-Einträge verwerfen (Squash)
- Architekturzustand entspricht dem Stand vor dem Commit dieses Befehls
- Danach: Trap / ISR — spekulative Arbeit war unsichtbar

### Notizen

Eleganz: Zukunftsberechnungen verwerfen, ohne RAM-Inkonsistenzen zu hinterlassen.

## Grenzen von OoO

Warum viele Cortex-M / MCU-Kerne ohne OoO bleiben:

- Transistoren: Renaming, Stations, ROB sind teuer
- Energie: ständiges Wakeup/Scheduling kostet — oft Watt statt  $\mu\text{W}$ /mW-Regime
- Determinismus: Laufzeiten schwerer vorherzusagen (Hard Real-Time)

### Notizen

Königsklasse für Desktop/Phone — für Knopfzellen-Sensoren oft ungeeignet.

## Drei Phasen — Kurzfazit

| Phase        | Ordnung      | Aufgabe                     |
|--------------|--------------|-----------------------------|
| Fetch/Decode | in-order     | lesen, renaming             |
| Execute      | out-of-order | Units auslasten             |
| Commit       | in-order     | sichtbar machen, Exceptions |

### Notizen

Anatomie eines High-End-Kerns in einem Satz: geordnet rein, parallel rechnen, geordnet raus.

## Fragen? — Out-of-Order

- Ziel des Register Renamings?
- Rolle der Reservation Stations?
- Warum ist der ROB für Interrupts/Exceptions unverzichtbar?

## Notizen

Fortsetzung: Branch Prediction — Sprünge, die selbst OoO-Kerne teuer treffen können.

## Control Hazards

- Datenabhängigkeiten: u. a. per Renaming entschärft
- Neuer Engpass: Kontrollfluss (`beq` und andere Branches)
- Beim Fetch des Branch: welcher nächste PC — Fall-through oder Target?
- Entscheidung oft erst nach dem Vergleich in EX (oder später in tiefen Pipelines)

## Notizen

Die Pipeline braucht jeden Takt neuen Nachschub — an der Weggabelung fehlt das Vergleichsergebnis noch.

## Kosten eines Pipeline-Flush

- Warten auf die ALU: wenige Bubbles — bei 5 Stufen oft erträglich
- Moderne High-Performance-Kerne: deutlich tiefere Pipelines (Größenordnung: viele Stufen), u. a. für hohe Taktraten
- Falscher Weg → Flush vieler spekulativer Instruktionen
- Misprediction Penalty: oft im Bereich vieler Takte — teuer für IPC

## Notizen

Tiefe Pipelines erhöhen die Branch Penalty: falsch geraten heißt viele Arbeitseinheiten verwerfen.

### Misprediction: Zahlenbeispiel

- 20 % der Befehle sind Branches
- Misprediction-Rate 10 %, Penalty 15 Takte
- Basis-CPI = 1 (ideale Pipeline ohne andere Stalls)
- CPI-Aufschlag:  $0,20 \times 0,10 \times 15 = 0,30$
- Effektives CPI  $\approx 1,30 \rightarrow$  ca. 23 % langsamer als ideal

#### Notizen

Klausurtypisch: Branch-Anteil  $\times$  Fehlrate  $\times$  Penalty aufs CPI aufschlagen.  $1,30/1 = 1,30$  Takte pro Befehl  $\Rightarrow$  Durchsatz  $1/1,30 \approx 0,77$  — rund 23 % langsamer.

### Branch Prediction

- Nicht warten — spekulativ den wahrscheinlichen Weg füllen
- Vorhersage typisch schon beim Fetch
- Treffer: praktisch keine Branch-Latenz
- Fehlprognose: Flush / Misprediction Penalty
- Ziel: hohe Accuracy (in der Praxis oft deutlich über 90 %)

#### Notizen

Glaskugel der Architektur: richtig geraten  $\rightarrow$  Pipeline bleibt voll; falsch  $\rightarrow$  teurer Rollback.

### Statische Vorhersage

Einfache fest verdrahtete Heuristiken:

- Rückwärtssprung oft „taken“ (typisch Schleifenende)
- Vorwärtssprung oft „not taken“ (häufig seltene Fehlerpfade)

Hilfreich, aber für sehr tiefe Pipelines oft zu ungenau.

#### Notizen

90er-Heuristik: hinten = Schleife, vorne = seltenes `if` — gut, aber nicht genug für 15+ Stufen.



## Dynamische Vorhersage

- Lernen zur Laufzeit aus der Branch-Historie
- Muster: was zuletzt passierte, passiert oft wieder
- Lange Schleifen: nach wenigen Iterationen fast immer korrektes „taken“

### Notizen

Maschinenlernen auf unterster Ebene: pro Sprungstelle Historie im Silizium.

## Branch History Table (BHT)

- Kleiner schneller Speicher (oft SRAM) für Predictions
- Index: typisch untere PC-Bits der Branch-Adresse
- Parallel zum Fetch: Eintrag liefert Taken/Not-Taken-Historie

### Notizen

Notizbuch der if-Abfragen: Hausnummer = PC, Eintrag = letztes Verhalten.

## 1-Bit-Prädiktor

- Ein Bit: zuletzt Taken (1) oder Not Taken (0)
- Schwachpunkt: verschachtelte Schleifen
- Schleifenende kippt auf Not Taken → nächster Eintritt oft sofort falsch geraten
- Zwei Fehlprognosen pro äußerem Durchlauf sind typisch

### Notizen

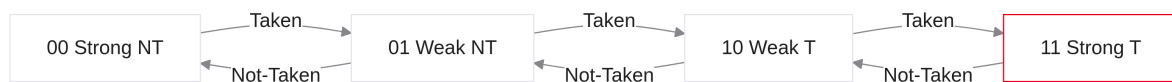
Zu reaktiv: ein Abbruch überschreibt die „Schleifen-Meinung“ zu aggressiv.

## 2-Bit saturierender Zähler

Industrie-Standard: Zustände 00–11

| Code | Bedeutung          | Vorhersage |
|------|--------------------|------------|
| 00   | Strongly Not Taken | Not Taken  |
| 01   | Weakly Not Taken   | Not Taken  |
| 10   | Weakly Taken       | Taken      |
| 11   | Strongly Taken     | Taken      |

Taken: +1 (Sättigung bei 11); Not Taken: −1 (Sättigung bei 00).



### Notizen

State Machine mit Schwelle in der Mitte: 10/11 → Taken, sonst Not Taken.

## Hysterese des 2-Bit-Zählers

- Lange Schleife → oft 11 (Strongly Taken)
- Einmaliges Ende: 11 → 10 — Vorhersage bleibt Taken
- Nächster Schleifeneintritt: oft sofort korrekt; Zähler steigt wieder
- Meinungswechsel erst nach zwei widersprechenden Ergebnissen in Folge

### Notizen

Trägheit spart Fehlprognosen bei typischen Schleifenabbrüchen.

### Branch Target Buffer (BTB)

- Taken allein reicht nicht — Zieladresse muss früh bekannt sein
- Decode/Add für Offset oft zu spät für den nächsten Fetch
- BTB: Cache mit (u. a.) zuletzt beobachteter Target-Adresse
- IF: Prediction + bei Taken sofortiger Target aus dem BTB

#### Notizen

GPS-Gedächtnis: nicht nur links/rechts, sondern die Zielkoordinaten ohne Wartezeit.

### Globale Historie (Korrelation)

- Branches korrelieren oft im C-Kontrollfluss
- Global History Register (GHR): Bitmuster der letzten  $N$  Outcomes
- Indexierung der Prediction-Tabellen mit lokaler + globaler Historie
- Erkennt Muster über einzelne Sprungstellen hinweg

#### Notizen

Nicht nur „dieser beq“, sondern „was tat das Programm kurz davor?“.

### Fortgeschrittene Prädiktoren

- High-End-Kerne: deutlich komplexere Modelle (u. a. Perceptron-ähnlich)
- Lernen von Gewichtungen aus der Historie in Hardware
- Praxis: sehr hohe Accuracy bei vielen Workloads — selten, aber teuer bei Misses

#### Notizen

Kein ChatGPT im Kern — spezialisierte, winzige Lernhardware für Sprungmuster.

## Fragen? — Branch Prediction

- Warum sind Misses bei tiefen Pipelines teuer?
- Wie hilft der 2-Bit-Zähler bei verschachtelten Schleifen?
- Rolle des BTB?

### Notizen

Als Nächstes: Spekulation = Prediction × OoO — und warum Caches Seitenkanäle öffnen können.

## Spekulative Ausführung

- Kombination: Branch Prediction + Out-of-Order
- Predictor sagt z. B. „Taken“ → spekulativer Pfad wird gestartet (Issue)
- ALUs rechnen und laden bereits — bevor der Branch architektonisch bestätigt ist

### Notizen

Zenit der Performance: rechnen auf Pfaden, die logisch noch unsicher sind.

## Rechnen auf Verdacht

- Best Case: Prediction korrekt → Arbeit schon (fast) fertig im ROB
- Speicherlatenzen werden hinter Spekulation versteckt
- IPC kann stark steigen, wenn Units nicht idle auf den Branch warten

### Notizen

Während der Branch noch „unterwegs“ ist, läuft der vorhergesagte Pfad schon.

## Rollback bei Misprediction

- Falscher Pfad: spekulative Loads/Rechnungen verwerfen
- ROB hält Ergebnisse zurück — kein vorzeitiges Commit in Architekturzustand
- Squash des spekulativen ROB-Teils, dann korrekten Pfad nachfüllen

### Notizen

Sicherheitsnetz aus Teil 1: spekulieren darf, sichtbar wird erst nach Bestätigung.

## Spectre und Meltdown (2018)

- Lange Annahme: Fehlspekulation sei für Software unsichtbar
- Meltdown: spekulativ über Privilege-Grenzen (z. B. Kernel-Daten) — vor allem bestimmte ältere Designs
- Spectre: spekulativ über fehlvorhergesagte Branches / manipulierte Predictor-Zustände
- Gemeinsam: Architekturzustand wird zurückgerollt — Cache-/Timing-Spuren können bleiben

### Notizen

Zwei verwandte, aber unterschiedliche Klassen: Privilege-Bypass (Meltdown) vs. Branch-/Predictor-Missbrauch (Spectre).

## Seitenkanal über den Cache

Prinzip (Spectre-artig, stark vereinfacht):

1. Predictor wird so beeinflusst, dass eine Bounds-Prüfung fehlgeht
2. Spekulativ: Zugriff außerhalb des erlaubten Index
3. Geheimnis beeinflusst Folgezugriffe (Cache-Zustand)
4. Rollback löscht Register — Cache-Zustand bleibt messbar

### Notizen

Kein „Passwort im Register behalten“ — das Timing des Caches verrät Indizes.

## Timing als Orakel

- Angreifer misst Zugriffszeiten (Flush+Reload / ähnliche Muster)
- Schneller Hit  $\Rightarrow$  Zeile war im Cache
- Aus der getroffenen Position lässt sich Information über spekulativ genutzte Werte rekonstruieren
- Klassischer mikroarchitektonischer Side Channel

## Notizen

Stoppuhr statt direktem Leserecht — Architektur verbietet, Mikroarchitektur leakt.

## Mitigationen

- Software: härtere Isolation (z. B. KPTI) — oft mit Leistungskosten
- Microcode / Fences: Spekulation an kritischen Stellen einschränken
- Neuere Generationen: strengere Isolation spekulativer Cache-Effekte in Hardware
- Trade-off: Sicherheit vs. IPC

## Notizen

Patches und Fences kosten Performance; langfristig hilft Silizium-Design.

## Ausblick Labor: Compiler hilft der Pipeline

- Hardware kompensiert viel (OoO, BHT) — Compiler kann Hazards und Branches reduzieren
- GCC/Clang: Optimierungsstufen `-O0 ... -O3`
- Ziel: bessere IPC, weniger Stalls und teure Control Hazards

## Notizen

Kein direkter ROB-Zugriff — aber Maschinencode so formen, dass die Pipeline „glücklich“ wird.

### Was -O3 typisch tut

- -O0: zeilennahe Übersetzung — gut debugbar, schlecht für Pipelines
- -O3: globale Analyse, Umsortierung, aggressivere Transformationen
- Fokus: Latenz verstecken, Branches reduzieren, ILP freilegen

### Notizen

Derselbe Algorithmus, anderer Assembler — Laufzeit kann stark differieren.

### Trick: Instruction Scheduling

- Problem:  $\perp_w \rightarrow$  abhängiger Nutzer  $\rightarrow$  Load-Use-Stall
- Compiler schiebt unabhängige Instruktionen in die Lücke (statisches Scheduling)
- C-Reihenfolge  $\neq$  optimale Maschinenreihenfolge

### Notizen

Blasen in der Pipeline mit nützlicher Arbeit füllen.

### Trick: Loop Unrolling

- Enge Schleifen: viele Branches relativ zur Nutzarbeit
- Unrolling: Iterationen ausrollen  $\rightarrow$  weniger `beq`, mehr gerader Code
- Trade-off: größere Code-Größe (I-Cache) vs. weniger Control Hazards

### Notizen

Kontrolle gegen Branches — Flash/I-Cache zahlen den Preis.

**Tool: Compiler Explorer (Godbolt)**

- Web-Tool: C links, Assembler rechts (live)
- Farbzuordnung C-Zeile → Instruktionen
- Ideal, um `-O1` vs. `-O3` und RISC-V-Output zu vergleichen

**Notizen**

Röntgenblick für Embedded- und Performance-Entwicklung.

**Laboridee: Godbolt und Messung**

Geplant (Details im Übungsblatt):

- Target z. B. `rv32gc` in Godbolt
- Umschalten `-O1` / `-O3`: Unrolling und Scheduling sichtbar machen
- Optional: Zyklusmessung in Renode/Ripes — Laufzeit/CPI-Effekt belegen

**Notizen**

Sehen (Assembler) und messen (Zyklen) — derselbe C-Algorithmus, andere Pipeline-Last.

**Zusammenfassung Block 7**

- Skalare Pipeline: Ideal oft  $IPC \leq 1$
- Superskalar: mehrere Units / Issues pro Takt
- OoO: Renaming + ROB für dynamisches Scheduling und präzise Exceptions
- Branch Prediction (u. a. 2-Bit, BTB): Flushes vermeiden
- Spekulation beschleunigt — Cache-Seitenkanäle (Spectre/Meltdown) als Kehrseite
- Compiler (`-O3`): Scheduling und Unrolling unterstützen die Hardware

**Notizen**

Moderne Kerne: vorhersagend, spekulativ, superskalar — Semantik nach außen dennoch „geordnet“.



**Evolution: MCT1 → MCT3**

- MCT1: klare In-Order-Pipeline, begrenztes ILP
- MCT3 bisher: Caches, DMA, Bus-Mastering
- Heute: physische Register, OoO, Prediction, Spekulation
- Brücke zur realen High-Performance- und zur energieeffizienten MCU-Welt

**Notizen**

Vom Addierer zum spekulativen Kern — Konzepte von Grund auf aufgebaut.

**Ausblick und Abschluss**

- Übungsblatt: Godbolt — 00/03, Scheduling, Unrolling
- Optional: Zyklusvergleich in der Simulation

Vielen Dank — viel Erfolg beim Experimentieren.

**Notizen**

Wer Assembler-Optimierungen lesen kann, programmiert näher an der Hardware. Viel Erfolg!